Practical-9
Exception Handling, Multithreading, And Unnamed Variable.

Q1.Write a Java program that reads an integer from the user. If the input is not an integer, handle the InputMismatchException.

Code:

```
import java.util.InputMismatchException;
import java.util.Scanner;


public class q1 {     public static void
main(String[]args)
    {
        Scanner sc = new Scanner(System.in);
try{
        System.out.println("Enter the Number:");

        int x=sc.nextInt();
        System.out.println("You entered: " + x);
        } catch(InputMismatchException e){
            System.out.println("its not  integer value..");
        }
finally{
sc.close();
        }
    }
}
```

Ans:
Enter the Number:
f its not integer
value..

Q2.Write a program that reads an array of integers and attempts to access an invalid index. Handle ArrayIndexOutOfBoundsException.

Code:

```
import java.util.Scanner;

public class q2 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
```

```
            System.out.print("Enter the size of the array: ");
int size = scanner.nextInt();

            int[] numbers = new int[size];

            System.out.println("Enter " + size + " integers:");
            for (int i = 0; i < size; i++) {
numbers[i] = scanner.nextInt();             }

            System.out.print("Enter an index to access: ");
int index = scanner.nextInt();

            System.out.println("Value at index " + index + " is: " + numbers[index]);

        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Invalid index. The index is outside the array bounds.");
        } finally {
            scanner.close();
        }
    }
}
```

Ans:
Enter the size of the array: 4 Enter
4 integers:
23
5
67
455
Enter an index to access: 5
Error: Invalid index. The index is outside the array bounds.

Q3.Write a program that divides two numbers. Throw an ArithmeticException using throw if the divisor is zero.

Code:

```
import java.util.Scanner;

public class q3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter the numerator: ");
int numerator = scanner.nextInt();

            System.out.print("Enter the denominator: ");
int denominator = scanner.nextInt();
```

```
        if (denominator == 0) {                throw new
ArithmeticException("Division by zero is not allowed.");
        }

        int result = numerator / denominator;
        System.out.println("Result: " + result);

    } catch (ArithmeticException e) {
        System.out.println("Error: " + e.getMessage());
    } finally {
        scanner.close();
    }
  }
}
```

Ans:
Enter the numerator: 56
Enter the denominator: 0
Error: Division by zero is not allowed.

Q4.Write a method calculate() that divides two numbers and declares throws ArithmeticException. Call it from main and handle the exception there.

Code:

```
import java.util.Scanner;

public class q4 {

  public static int calculate(int numerator, int denominator) throws ArithmeticException {
if (denominator == 0) {            throw new ArithmeticException("Division by zero is not
allowed.");
    }
    return numerator / denominator;
  }

  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    try {
        System.out.print("Enter numerator: ");
int num = scanner.nextInt();

        System.out.print("Enter denominator: ");
int den = scanner.nextInt();

        int result = calculate(num, den);
        System.out.println("Result: " + result);
```

```
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Ans:
Enter numerator: 56
Enter denominator: 34
Result: 1

Q5.Write a Java program to calculate the square root of a number with proper exception handling. Create a method which will throws ArithmeticException. Inside the method: Check if the number is negative if yes then use the throw keyword to throw an ArithmeticException with an appropriate message. The input is valid, then return the square root of the number. Inside the main method, Accept a number from the user. Call the findSquareRoot() method inside a try block to Handle the exception using a catch block. Display either the result or the error message

Code:

```
import java.util.Scanner; class SquareRootDemo {     static double
findSquareRoot(double num) throws ArithmeticException {         if (num <
0) {             throw new ArithmeticException("Negative number not
allowed");
        }
        return Math.sqrt(num);
    }
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter a number: ");
        double num = sc.nextDouble();
try {
            double result = findSquareRoot(num);
            System.out.println("Square Root = " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

Ans:
Enter a number: -45
Error: Square root of a negative number is not allowed.

Q6.Create a custom exception called AgeException and write a program that throws this exception if the user enters an age less than 18.

Code:

```
import java.util.Scanner; class
AgeException extends Exception {
AgeException(String message) {
      super(message);
   }
}
public class q6 {     public static void
main(String[] args) {
      Scanner sc = new Scanner(System.in);
System.out.print("Enter age: ");
      int age = sc.nextInt();
      try {
         if (age < 18) {               throw new AgeException("Age
must be 18 or above");
         }
         System.out.println("Access granted");
      }
      catch (AgeException e) {
         System.out.println("Error: " + e.getMessage());
      }
      sc.close();
   }
}
```

Ans:

```
Enter age: 17
Error: Age must be 18 or above
```

Q7.Write a program to create a thread by extending the Thread class. Override the run() method. Display a message inside the thread execution. Start the thread from the main() method.

Code:

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running...");
   }
}
public class Main7 {     public static void
main(String[] args) {        MyThread t =
new MyThread();
      t.start();
   }
```

```
}
```

Ans:

Thread is running…

Q8.Write a program to create a thread using the Runnable interface. Implement the run() method. Pass the runnable object to a Thread object. Start the thread and display a message.

Code:

```
class MyRunnable implements Runnable {
public void run() {
      System.out.println("Thread is running using Runnable...");
   }
}
public class Main8 {    public static void
main(String[] args) {
      MyRunnable obj = new MyRunnable();
Thread t = new Thread(obj);
      t.start();
   }
}
```

Ans:

Thread is running using Runnable…

Q9.**Write a program that creates two threads, one thread prints the multiplication table of 5 and another thread prints the multiplication table of 10. Implementing the program without synchronization and observer the output. Then implement with synchronization and observe the output.**

Code:

```
class TableThread extends Thread {
   int num;
   TableThread(int num) {
      this.num = num;
   }
   public void run() {        for
(int i = 1; i <= 5; i++) {
         System.out.println(num + " x " + i + " = " + (num * i));
      }
   }
} public class Asyn9 {    public static void
main(String[] args) {       TableThread t1 =
new TableThread(5);       TableThread t2 =
new TableThread(10);
```

```
        t1.start();
t2.start();
    }
}
```

Ans:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
```

Syn9.java

```
class Table {     synchronized
void printTable(int num) {          for (int i =
1; i <= 5; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }
    }
}
class MyThread extends Thread {
    Table t;
    int num;
    MyThread(Table t, int num) {
        this.t = t;
        this.num = num;
    }
    public void run() {
        t.printTable(num);
    }
}
public class Syn9 {
    public static void main(String[] args) {
        Table obj = new Table();
        MyThread t1 = new MyThread(obj, 5);
        MyThread t2 = new MyThread(obj, 10);
        t1.start();
        t2.start();
    }
}
```

Ans:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50

Q 10.**Write a Java program to print the multiplication table of 5 using an unnamed variable (_).**

Code:

```
import java.util.List; public class Q10{
public static void main(String[] args) {

      int count = 0;
      List<String> myList = List.of("a", "b", "c", "d", "e","a", "b", "c", "d", "e");
count++;
      for(String _ : myList) {
         System.out.println(count++ * 5);
      }
   }
}
```

Ans:
5
10
15
20
25
30
35
40
45
50

Q11.**Write a program to demonstrate garbage collection.**

Code: class

```
GarbageDemo {
   protected void finalize() {
      System.out.println("Garbage collected object");
   }
   public static void main(String[] args) {
      GarbageDemo obj1 = new GarbageDemo();
      GarbageDemo obj2 = new GarbageDemo();
      obj1 = null;
obj2 = null;
System.gc();
      System.out.println("End of main method");
   }
}
```

Ans:

End of main method
Garbage collected object
Garbage collected object